

Text Vectorization I: Basic vectorization techniques

Natural Language Processing
Master in Applied Artificial Intelligence

Lorena Calvo Bartolomé
lcalvo@pa.uc3m.es

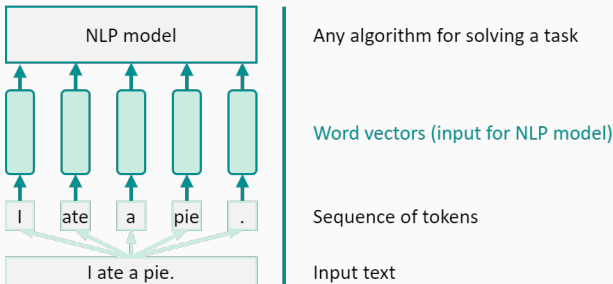
November 10, 2023

Department of Signal Theory and Communications
Universidad Carlos III de Madrid

1. Introduction
2. Basic vectorization techniques
3. Motivation to Word Embeddings
4. Text preprocessing

How to represent words?

- Humans represent things/concepts through language:
 - E.g., 🚗 ⇔ car, 🔔 ⇔ bell, ✂ ⇔ scissors, etc.
- But ML models “see” data in a different way than humans do
 - We can easily understand the text “*I ate a pie*”, but our models cannot because they require **feature vectors**



How to represent words?

The way we represent words as input to any of our models is the most important common denominator across all NLP tasks

Word vectors

Representations of words that can be fed into an NLP model

How do we get these word /document representations?

- One-hot encoding
- Bag-of-Words
- TF-IDF
- Distributional semantics based (Word2Vec, Glove...)
- Contextual Word Embeddings

Table of Contents

- 1. Introduction
- 2. Basic vectorization techniques
 - 2.1 One-hot encoding
 - 2.2 The Bag of Words model
 - 2.3 TF-IDF representation
- 3. Motivation to Word Embeddings
- 4. Text preprocessing

One-hot encoding model

One-hot vector

Let V be the vocabulary, represent every word as an $\mathcal{R}^{|V| \times 1}$ vector with all 0s and one 1 at the index of that word in vocabulary V

	Index				
Token	1	2	3	4	
car	1	0	0	0	w^{car}
board	0	1	0	0	w^{board}
dock	0	0	1	0	w^{dock}
pie	0	0	0	1	w^{pie}

	Index					
Token	1	2	3	4		V
car	1	0	0	0	...	0
board	0	1	0	0	...	0
dock	0	0	1	0	...	0
pie	0	0	0	1	...	0
...	...	...	...	...	...	...
apple	0	0	0	0	...	1

Problems

1. One-hot encoding does not scale well with dictionary size
2. One-hot vectors do not capture meaning, nor give any notion of similarity

$$(w^{car})^T w^{board} = 0 = (w^{board})^T w^{pie}$$

Bag-of-Words model

Bag-of-Words document representation

Count the number of appearances of each word (in the vocabulary of all documents) in a given document

Document One	Token	Doc 1	Doc 2	Doc 3
Can you can a car as a canner can can a can?	a	3	0	0
	as	1	0	0
	ate	0	1	0
	bean	0	1	0
Document Two He ate the whole can of beans.	can	6	1	1
	canner	1	0	0
	he	0	1	1
	of	0	1	0
Document Three He can read.	read	0	0	1
	the	0	1	0
	whole	0	1	0
	you	1	0	0

Problem: Any information about the order or structure of words in the document is discarded

→ The model is only concerned with whether known words occur in the document, not where in the document

» The BoW representation of each word in a document is obtained by summing up all OHE vectors of that word in such a document

Term frequency - Inverse document frequency (TF-IDF)

TF-IDF representation

Give a high value for a given term in a given document if that term occurs often in that particular document and very rarely anywhere else

- **Term frequency:** Number of instances of a given word in a document

$$TF(w, d) = \frac{BoW(w, d)}{\#words\ in\ d}$$

- **Inverse document frequency:** Logarithm of the number of documents in the collection divided by the number of documents with a specific word

$$IDF(w) = \log \frac{\#docs}{\#docs\ with\ term\ w}$$

$$TF - IDF(w, d) = TF(w, d) \times IDF(w)$$

Emphasizes the words that
appear in a reduced nr of docs

Term frequency - Inverse document frequency (TF-IDF)

Document One	Token	BoW Doc 1	BoW Doc 2	BoW Doc 3	TF(w,d ₁)	TF(w,d ₂)	TF(w,d ₃)	IDF(w)	TF-IDF Doc 1	TF-IDF Doc 2	TF-IDF Doc 3
Can you can a car as a canner can can a can?	a	3	0	0	0,25	0	0	0,48	0,12	0	0
	as	1	0	0	0,08	0	0	0,48	0,04	0	0
	ate	0	1	0	0	0,14	0	0,48	0	0,07	0
Document Two He ate the whole can of beans.	bean	0	1	0	0	0,14	0	0,48	0	0,07	0
	can	6	1	1	0,5	0,14	0,33	0	0	0	0
	canner	1	0	0	0,08	0	0	0,48	0,04	0	0
	he	0	1	1	0	0,14	0,33	0,18	0	0,03	0,06
	of	0	1	0	0	0,14	0	0,48	0	0,07	0
Document Three He can read.	read	0	0	1	0	0	0,33	0,48	0	0	0,16
	the	0	1	0	0	0,14	0	0,48	0	0,07	0
	whole	0	1	0	0	0,14	0	0,48	0	0,07	0
	you	1	0	0	0,08	0	0	0,48	0,04	0	0

- BoW creates a set of vectors containing the documents' word occurrences, while TF-IDF contains information on both the more and less important words
- BoW vectors are easy to interpret; TF-IDF usually performs better in machine learning models

Table of Contents

1. Introduction
2. Basic vectorization techniques
3. Motivation to Word Embeddings
4. Text preprocessing

- The **BoW** model is very limited, it represents the frequencies of each word in a piece of text and nothing else
- **TF-IDF** is based on the BoW model, therefore it does not capture position in texts, semantics, co-occurrences in different documents, etc.
- **One-hot encoded vectors** also presents severe limitations
 - Vectors become more sparse as the vocabulary grows
 - Every word is orthogonal to each other ⇒ **They do not capture meaning!**

But how do we know what meaning is?

Do you know what the word **tezgüino** means?

(We hope you do not)



Now look how this word is used in different contexts:

A bottle of **tezgüino** is on the table.

Everyone likes **tezgüino**.

Tezgüino makes you drunk.

We make **tezgüino** out of corn.



Can you understand what **tezgüino** means?

Now look how this word is used in different contexts:

A bottle of **tezgüino** is on the table.

Everyone likes **tezgüino**.

Tezgüino makes you drunk.

We make **tezgüino** out of corn.



Tezgüino is a kind
alcoholic beverage
made from corn.



With context, you can understand the meaning!



How did you do this?

What others words fit into these context?

- ① A bottle of _____ is on the table.
- ② Everyone likes _____.
- ③ _____ makes you drink.
- ④ We make _____ out of corn.



	1	2	3	4	...
tezgüino	1	1	1	1	
loud	0	0	0	0	
motor oil	1	0	0	1	
tortillas	0	1	0	1	
wine	1	1	1	0	

← context

← row show contextual properties: 1 if a words can appear in the context. 0 if not

- ① A bottle of _____ is on the table.
- ② Everyone likes _____.
- ③ _____ makes you drink.
- ④ We make _____ out of corn.



	1	2	3	4	...
tezgüino	1	1	1	1	
loud	0	0	0	0	
motor oil	1	0	0	1	
tortillas	0	1	0	1	
wine	1	1	1	0	

This is the **distributional hypothesis**

rows are
similar



meanings of the
words are similar

Distributional semantics

Distributional hypothesis

Words which frequently appear in similar contexts have similar meaning

» “You shall know a word by the company it keeps” (J. R. Firth, 1957)

💡 Main idea

“To capture meaning” ⇔ “to capture context”

» We need to encode information about word contexts into the word representation

The **context of a word** appearing in a text is the set of nearby words

...and the little page was kept busy **running** on errands until past midnight.

...at least when he was **running** for president.

...the secretive systems of bunkers and tunnels **running** beneath major cities that...

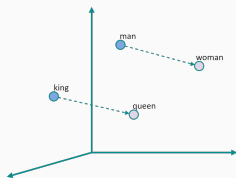
These **context words** will represent **running**

Word meaning as a vector

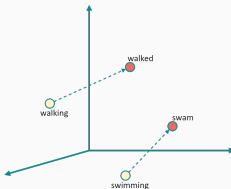
Word embeddings

Type of word representation that allows words with similar meaning to have close representations

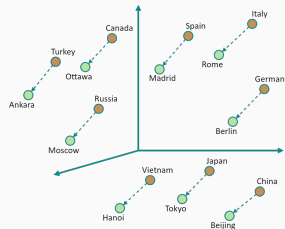
- For each word, we will have a dense vector chosen so that it is similar to other word vectors that appear in a similar context
 - » The similarity is measured as the vector **dot** (scalar) **product**



Male-Female



Verb Tense



Country-Capital

- Embedding Projector 

Table of Contents

1. Introduction
2. Basic vectorization techniques
3. Motivation to Word Embeddings
4. Text preprocessing

💡 Main idea

Text data derived from natural language is **unstructured** and **noisy**

- » In order to have a **useful representation** with any text vectorization technique, some preprocessing steps are normally required



Other additional steps:

- Removing punctuation like .,!@#\$*%
- Remove URLs
- Ad-hoc steps in the NLP problem we are dealing with



Process of **segmenting a text into words**, referred as *tokens*.

- » This procedure will often also break off punctuation symbols (commas, periods, etc.), phrases, and other possible meaningful elements from the text, such as separate tokens.

Example:

We are given the following Nietzsche quote to tokenize:

"You have your way. I have my way. As for the right way, the correct way, and the only way, it does not exist." (**Friedrich Nietzsche.**)

The resulting tokenized text will look as follows:

'You', 'have', 'your', 'way', ',', 'I', 'have', 'my', 'way', ',', 'As', 'for', 'the', 'right', 'way',
,, 'the', 'correct', 'way', ,, 'and', 'the', 'only', 'way', ,, 'it', 'does', 'not', 'exist', ,,



Process that aims at **collapsing all semantically equivalent words** into a unique representative one.

- Different forms of a word: organize, organizes, organizing, ... → organize
- Derivationally related words with similar meanings: democracy, democratic, democratization, ... → democracy

Stemming vs lemmatization:

- **Stemming** chops of the ends of words using a list of suffixes
 - **Lemmatization** usually refers to doing things properly with a vocabulary and morphological analysis of words, aiming to return the base or dictionary form (lemma) of a word.
 - A practical difference is that lemmatizers output complete words.
- » If using case sensitive implementations, we need to lowercase the tokens as a preliminary step

Stemming vs lemmatization:

Example:

Continuing with the tokenized quote from the Nietzsche's example:

- **Stemmed text:**

'you', 'have', 'your', 'way', 'i', 'have', 'my', 'way', '**as**', 'for', 'the', 'right', 'way', 'the', 'correct', 'way', 'and', 'the', '**onli**', 'way', 'it', 'doe', 'not', 'exist'

- **Lemmatized text:**

'you', 'have', 'your', 'way', 'i', 'have', 'my', 'way', '**a**', 'for', 'the', 'right', 'way', 'the', 'correct', 'way', 'and', 'the', '**only**', 'way', 'it', 'doe', 'not', 'exist'

Other homogenization tasks

Part-of-Speech Tagging

Process of assigning a word to its grammatical category (noun, verb, adverb,,) in order to understand its role within the sentence.

- It provides the contextual information during the lemmatization process to filter grammatical differences.

Example:

Continuing with the tokenized quote from the Nietzsche's example:

('you', 'PRP'), ('have', 'VBP'), ('your', 'PRP'), ('way', 'NN'), ('i', 'NN'), ('have', 'VBP'), ('my', 'PRP'), ('way', 'NN'), ('as', 'IN'), ('for', 'IN'), ('the', 'DT'), ('right', 'JJ'), ('way', 'NN'), ('the', 'DT'), ('correct', 'JJ'), ('way', 'NN'), ('and', 'CC'), ('the', 'DT'), ('only', 'JJ'), ('way', 'NN'), ('it', 'PRP'), ('does', 'VBZ'), ('not', 'RB'), ('exist', 'VB')

N-gram detection

Identification of groups of words that occasionally appear together, and have an inherent semantic value, e.g., Machine learning, big data, etc.



Process that aims to eliminate all words that are irrelevant for determining the semantically meaning of a document:

- “*Stopwords*”, which are words that appear too often in very different contexts, but do not carry out useful semantic content, like articles or pronouns.
- Both very rare and very common terms, i.e., words that appear only in a few documents and in a lot of documents inside the corpora, respectively.

Example:

The cleaned tokenized quote will look as follows:

'way', 'way', 'right', 'way', 'correct', 'way', 'way', 'doe', 'exist'

Lab exercise “Text Vectorization I”

- **Objective:** To provide a basic overview of the following aspects:
 - NLP preprocessing
 - Document BoW and TF-IDF representation
 - Utilization of the latter to solve a Text Classification task

Bibliography i



Alammar, J. (2019). The illustrated word2vec.
<https://jalammar.github.io/illustrated-word2vec/>



Aßenmacher, M. (2020). Modern approaches in natural language processing.
https://slds-lmu.github.io/seminar_nlp_ss20/



Eisenstein, J. (2018). Natural language processing. *Jacob Eisenstein*.



Godoy, D. V. (2022). *Deep learning with pytorch step-by-step, volume iii: Sequences & nlp*. Independently published.



Jeffrey Pennington, C. D. M., Richard Socher. (2014). Glove: Global vectors for word representation. *<https://nlp.stanford.edu/projects/glove/>*



Manning, C., Socher, R., Fang, G. G., & Mundra, R. (2017). Cs224n: Natural language processing with deep learning1.



Matthew E. Peters, M. I., Mark Neumann. (2018). A review of deep contextualized word representations. *<https://www.slideshare.net/shuntaroy/a-review-of-deep-contextualized-word-representations-peters-2018>*

Bibliography ii



Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*.



Mikolov, T., Sutskever, I., Chen, K., Corrado, G. S., & Dean, J. (2013). Distributed representations of words and phrases and their compositionality. *Advances in neural information processing systems*, 26.



Pennington, J., Socher, R., & Manning, C. D. (2014). Glove: Global vectors for word representation. *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*, 1532–1543.



Voita, E. (2020). NLP Course For You.
https://lena-voita.github.io/nlp_course.html



Zhang, A., Lipton, Z. C., Li, M., & Smola, A. J. (2021). Dive into deep learning. *arXiv preprint arXiv:2106.11342*.